

semmcci: Monte Carlo Confidence Intervals

Ivan Jacob Agaloos Pesigan

Installation

You can install the CRAN release of `semmcci` with:

```
install.packages("semmcci")
```

You can install the development version of `semmcci` from [GitHub](#) with:

```
if (!require("pak")) install.packages("pak")
pak::pkg_install("jeksterslab/semmcci")
```

Documentation

See [GitHub Pages](#) for package documentation.

Description

In the Monte Carlo method, a sampling distribution of parameter estimates is generated from the multivariate normal distribution using the parameter estimates and the sampling variance-covariance matrix. Confidence intervals for defined parameters are generated by obtaining percentiles corresponding to $100(1 - \alpha)\%$ from the generated sampling distribution, where α is the significance level.

Monte Carlo confidence intervals for free and defined parameters in models fitted in the structural equation modeling package `lavaan` can be generated using the `semmcci` package. The package has two main functions, namely, `MC()` and `MCStd()`. The output of `lavaan` is passed as the first argument to the `MC()` function to generate Monte Carlo confidence intervals. Monte Carlo confidence intervals for the standardized estimates can also be generated by passing the output of the `MC()` function to the `MCStd()` function. A description of the package and code examples are presented in Pesigan and Cheung (2024).

Example

A common application of the Monte Carlo method is to generate confidence intervals for the indirect effect. In the simple mediation model, variable **X** has an effect on variable **Y**, through a mediating variable **M**. This mediating or indirect effect is a product of path coefficients from the fitted model.

```
library(semmcci)
library(lavaan)
```

Data

```
n <- 1000
X <- rnorm(n = n)
M <- 0.50 * X + rnorm(n = n)
Y <- 0.25 * X + 0.50 * M + rnorm(n = n)
data <- data.frame(X, M, Y)
```

Model Specification

The indirect effect is defined by the product of the slopes of paths **X** to **M** labeled as **a** and **M** to **Y** labeled as **b**. In this example, we are interested in the confidence intervals of `indirect` defined as

the product of **a** and **b** using the `:=` operator in the **lavaan** model syntax.

```
model <- "  
  Y ~ cp * X + b * M  
  M ~ a * X  
  indirect := a * b  
  direct := cp  
  total := cp + (a * b)  
"
```

Model Fitting

We can now fit the model using the `sem()` function from **lavaan**.

```
fit <- sem(data = data, model = model)
```

Monte Carlo Confidence Intervals

The `fit` **lavaan** object can then be passed to the `MC()` function to generate Monte Carlo confidence intervals.

```
MC(fit, R = 20000L, alpha = 0.05)  
  
#> Monte Carlo Confidence Intervals  
#>      est      se      R  2.5% 97.5%  
#> cp      0.2409 0.0342 20000 0.1729 0.3077  
#> b       0.4736 0.0310 20000 0.4126 0.5345  
#> a       0.4740 0.0312 20000 0.4125 0.5353  
#> Y~~Y    0.9497 0.0425 20000 0.8663 1.0329  
#> M~~M    0.9760 0.0434 20000 0.8917 1.0606
```

```
#> X~~X      1.0006 0.0000 20000 1.0006 1.0006
#> indirect 0.2245 0.0209 20000 0.1848 0.2668
#> direct    0.2409 0.0342 20000 0.1729 0.3077
#> total     0.4654 0.0342 20000 0.3977 0.5327
```

Standardized Monte Carlo Confidence Intervals

Standardized Monte Carlo Confidence intervals can be generated by passing the result of the `MC()` function to `MCStd()`.

Note: We recommend setting `fixed.x = FALSE` when generating standardized estimates and confidence intervals to model the variances and covariances of the predictors if they are assumed to be random.

```
fit <- sem(data = data, model = model, fixed.x = FALSE)
unstd <- MC(fit, R = 20000L, alpha = 0.05)
vcov(unstd)
```

#>	cp	b	a	Y~~Y	M~~M
#> cp	1.179094e-03	-4.685192e-04	-7.441211e-06	1.026785e-05	-9.930748e-06
#> b	-4.685192e-04	9.552765e-04	4.981236e-06	-6.919804e-06	-6.447384e-06
#> a	-7.441211e-06	4.981236e-06	9.803667e-04	8.330835e-07	4.269761e-06
#> Y~~Y	1.026785e-05	-6.919804e-06	8.330835e-07	1.794939e-03	3.563855e-06
#> M~~M	-9.930748e-06	-6.447384e-06	4.269761e-06	3.563855e-06	1.897191e-03
#> X~~X	-9.059217e-06	-2.243585e-06	5.681379e-06	1.457134e-05	1.364754e-05
#> indirect	-2.254298e-04	4.554786e-04	4.668563e-04	-2.917003e-06	-1.053369e-06
#> direct	1.179094e-03	-4.685192e-04	-7.441211e-06	1.026785e-05	-9.930748e-06
#> total	9.536643e-04	-1.304062e-05	4.594151e-04	7.350844e-06	-1.098412e-05
#>	X~~X	indirect	direct	total	
#> cp	-9.059217e-06	-2.254298e-04	1.179094e-03	9.536643e-04	

```
#> b      -2.243585e-06  4.554786e-04 -4.685192e-04 -1.304062e-05
#> a      5.681379e-06  4.668563e-04 -7.441211e-06  4.594151e-04
#> Y~~Y    1.457134e-05 -2.917003e-06  1.026785e-05  7.350844e-06
#> M~~M    1.364754e-05 -1.053369e-06 -9.930748e-06 -1.098412e-05
#> X~~X    2.032576e-03  1.245982e-06 -9.059217e-06 -7.813235e-06
#> indirect 1.245982e-06  4.381688e-04 -2.254298e-04  2.127390e-04
#> direct  -9.059217e-06 -2.254298e-04  1.179094e-03  9.536643e-04
#> total   -7.813235e-06  2.127390e-04  9.536643e-04  1.166403e-03
```

MCStd(unstd)

```
#> Standardized Monte Carlo Confidence Intervals
#>      est      se      R 0.05%  0.5%  2.5% 97.5% 99.5% 99.95%
#> cp      0.2047 0.0288 20000 0.1117 0.1291 0.1485 0.2604 0.2773 0.2981
#> b      0.4410 0.0267 20000 0.3510 0.3706 0.3874 0.4921 0.5088 0.5244
#> a      0.4327 0.0258 20000 0.3484 0.3654 0.3816 0.4817 0.4961 0.5150
#> Y~~Y    0.6855 0.0240 20000 0.6031 0.6219 0.6376 0.7310 0.7459 0.7621
#> M~~M    0.8128 0.0223 20000 0.7347 0.7538 0.7679 0.8543 0.8665 0.8786
#> X~~X    1.0000 0.0000 20000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000
#> indirect 0.1908 0.0166 20000 0.1407 0.1496 0.1590 0.2239 0.2338 0.2480
#> direct  0.2047 0.0288 20000 0.1117 0.1291 0.1485 0.2604 0.2773 0.2981
#> total   0.3955 0.0266 20000 0.3014 0.3252 0.3426 0.4460 0.4611 0.4804
```

References

Pesigan, I. J. A., & Cheung, S. F. (2024). Monte Carlo confidence intervals for the indirect effect with missing data. *Behavior Research Methods*, 56(3), 1678–1696. <https://doi.org/10.3758/s13428-023-02114-4>

R Core Team. (2026). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. Vienna, Austria. <https://www.R-project.org/>